MACHINE LEARNING INTERNSHIP

Assignment 2: Supervised, Ensemble & Unsupervised ML Algorithms

Setup Requirement

Install the following libraries (or use Google Colab where they are pre-installed):

- **pip install scikit-learn numpy pandas matplotlib seaborn**
- All datasets used in this assignment are **built into scikit-learn**. No external download required.

Task 1: Linear Regression & Logistic Regression

Part A: Linear Regression (Salary Prediction)

Build a Linear Regression model that predicts **salary** based on **years of experience** using the dataset below.

years_experience	salary
1.1	39343
1.3	46205
1.5	37731
2.0	43525
2.2	39891
2.9	56642
3.0	60150
3.2	54445
3.7	57189
4.0	63218
4.5	67938
5.0	75000
5.5	83088
6.0	91738
7.0	101302
8.0	113812
9.0	121872
10.0	122391

Steps to perform:

- Load the data into a Pandas DataFrame.
- Split into **training (80%) and testing (20%)**.
- Train a **LinearRegression** model.
- Print **MSE, RMSE, and R-squared score**.
- Plot the regression line on the training data.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Build DataFrame manually
data = {'years_experience':[1.1,1.3,1.5,2.0,2.2,2.9,3.0,3.2,3.7,
                            4.0,4.5,5.0,5.5,6.0,7.0,8.0,9.0,10.0],
        'salary':[39343,46205,37731,43525,39891,56642,60150,54445,
                  57189,63218,67938,75000,83088,91738,101302,113812,
                  121872,122391]}
df = pd.DataFrame(data)

X = df[['years_experience']]
y = df['salary']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
print("RMSE:", np.sqrt(mse))
print("R-squared:", r2_score(y_test, y_pred))

plt.scatter(X_train, y_train, color='blue', label='Training data')
plt.plot(X_train, model.predict(X_train), color='red', label='Regression line')
plt.title('Salary vs Years of Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()
```

Part B: Logistic Regression (Binary Classification)

Use the built-in **Breast Cancer Wisconsin dataset** from scikit-learn. Build a Logistic Regression model to classify tumours as **malignant (0)** or **benign (1)**.

Steps to perform:

- Load the dataset using **load_breast_cancer()**.
- Split into 80/20 train/test.
- Train a **LogisticRegression** model with **max_iter=5000**.
- Print: accuracy, confusion matrix, and classification report.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Task 2: Decision Tree, Random Forest & Ensemble Techniques

Use the built-in **Iris dataset** from scikit-learn. Build, evaluate, and compare three tree-based models.

Steps to perform:

- Load the Iris dataset using **load_iris()**.
- Split into 70/30 train/test.
- Train and evaluate the following 3 models. Print accuracy for each:
  1. **Decision Tree Classifier**
  2. **Random Forest Classifier** (n_estimators=100)
  3. **Gradient Boosting Classifier** (ensemble technique)
- Plot the trained Decision Tree using **plot_tree()** from sklearn.
- Fill the comparison table below in your submission:

Model	Accuracy	Strengths	Weaknesses (in your words)
Decision Tree			
Random Forest			
Gradient Boosting			

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
print("Decision Tree Accuracy:", accuracy_score(y_test, dt.predict(X_test)))

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print("Random Forest Accuracy:", accuracy_score(y_test, rf.predict(X_test)))

# Gradient Boosting (ensemble)
gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_train, y_train)
print("Gradient Boosting Accuracy:", accuracy_score(y_test, gb.predict(X_test)))

# Visualize Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(dt, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.title('Decision Tree on Iris dataset')
plt.show()
```

Task 3: KNN, Naive Bayes, SVM & Unsupervised K-Means

Continue using the **Iris dataset**. Compare multiple classifiers and try unsupervised clustering.

Part A: Compare KNN, Naive Bayes and SVM

Train and evaluate the following 3 classifiers on the same train/test split (70/30):

- **K-Nearest Neighbours (KNN)** with k = 5
- **Gaussian Naive Bayes**
- **Support Vector Machine (SVC)** with linear kernel

Model	Accuracy
KNN (k = 5)	
Naive Bayes	
SVM (linear)	

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# (Use the same X_train, X_test, y_train, y_test from Task 2)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print("KNN Accuracy:", accuracy_score(y_test, knn.predict(X_test)))

nb = GaussianNB()
nb.fit(X_train, y_train)
print("Naive Bayes Accuracy:", accuracy_score(y_test, nb.predict(X_test)))

svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
print("SVM Accuracy:", accuracy_score(y_test, svm.predict(X_test)))
```

Part B: Unsupervised Learning with K-Means

Apply **K-Means clustering** on the Iris feature data (ignore the labels) and compare the discovered clusters with the actual species labels.

Steps to perform:

- Apply **KMeans** with **n_clusters = 3**.
- Plot the clusters using sepal length (X) and sepal width (Y), with colour by cluster.
- Write a 100-word note comparing supervised (KNN / SVM / etc.) and unsupervised (K-Means) learning.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

km = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = km.fit_predict(X)  # X = full iris features

plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
plt.title('K-Means Clustering of Iris (k = 3)')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```

Deliverables

• Task 1: Linear Regression code with MSE / RMSE / R-squared output and regression line plot, plus Logistic Regression code with accuracy, confusion matrix, and classification report.

• Task 2: Decision Tree, Random Forest, and Gradient Boosting accuracies, the Decision Tree plot, and the completed comparison table.

• Task 3: KNN, Naive Bayes, and SVM accuracies in the comparison table, K-Means cluster plot, and a 100-word supervised vs unsupervised note.

How to Submit

1. Create the assignment in MS Word (.doc / .docx) or PDF format.

2. Include your full name, course / internship name, and assignment number at the top of the document.

3. Paste your Python code and clear screenshots of every output for each task.

4. Upload the completed file to your Google Drive.

5. Enable sharing access by setting the file to "Anyone with the link can view".

6. Copy the Google Drive link and submit it before the assignment deadline through the official submission portal.